Business Process Performance Prediction on a Tracked Simulation Model

Andrei Solomon
4700 Keele Street
York University
Toronto, Canada
asolomon@cse.yorku.ca

Marin Litoiu
4700 Keele Street
York University
Toronto, Canada
mlitoiu@yorku.ca

ABSTRACT

Business processes need to achieve key performance indicators with minimum resources in changing operating conditions. Changes include hardware and software failures, load variation and variations in user interaction with the system. By incorporating simulation in the prediction model it is possible to predict with more confidence system performance degradations. We present our dynamic predictive model which uses forecasting techniques on historical process performance estimates for business process optimization. The parameters of the simulation model are estimates tuned at run-time by tracking the system with a particle filter.

Categories and Subject Descriptors

D.2.11 [**Software Engineering**]: Software Architectures—*Service-oriented architecture (SOA)*

General Terms

Performance, Simulation, Prediction

Keywords

BPM, ARIMA, Box-Jenkins, linear regression, prediction, simulation, estimation, optimization, business process, particle filter

1. INTRODUCTION

A business process has to achieve Key Performance Indicators (KPIs) to satisfy company business objectives. Although the process is designed and deployed to do so, based on the initial assumptions and requirements, changing operating conditions triggers the need for further evolution of the process.

Understanding the effect of changing operating conditions on the performance of the business process is essential for successful evolution in the context of the process life cycle.

PESOS'11, May 23-24, 2011, Waikiki, Honolulu, HI, USA

Tracing the execution of instances and analyzing the data can allow for quantitative evaluation of different change decisions [22, 19]. Simulation is appropriate when trying to gain insight in alternative situations and when experimentation is too expensive or even dangerous.

For satisfying Service Level Agreements (SLAs) it is important to predict possible breaches before they occur. This allows time for on-line and off-line corrective measures. At the same time, a process optimization needs forecasted KPIs to assess the effect of changes on the future performance. Having accurate forecasts about the performance of the system based on an on-line fine-tuned simulation model also means more accurate decisions at any point in time. In current industrial practice, for a deployed business process, the Business Process Monitor can present real time KPIs to the user through dashboards[14]. The dashboard also predicts trends in KPIs based solely on simple interpolation of their historical values, by looking at each KPI in isolation. However, this approach has limitations: by not taking into account the correlation among KPIs, it leads to prediction errors and wrong decisions.

This paper is concerned with building an accurate prediction model that can be used on line for business process optimization. The prediction model correlates the KPIs and the process variables through a simulation model. Input variables in the simulation model are estimated and then forecasted using different prediction models. Then the forecasted inputs are used by the simulator to calculate the forecasted KPIs. We also propose a feedback based evolution architecture that can tune a business process based on a simulation. Together with an estimator and an on-line monitor, the simulator accurately matches the execution of the real process. In sync with the underlying process and integrated with modeling tools, it accelerates the adaptation through the process life cycle [26]. This paper relies on estimation techniques discussed in [24].

The estimated states generated sequentially over time capture temporal behaviour, complex arrval patterns, seasonal (cyclical) patterns or even aperiodic components of the pattern. A *state predictor* uses an Auto-Regressive Moving Average (ARIMA) model as the baseline to identify, evaluate and forecast the behavior of a time series from historical process estimations alone. The output is a forecasted estimated state which is used as input for our simulation model, in order to produce the forecasted KPIs (e.g. end to end process duration). By simulating the forecasted estimates we take into account the structure of the process and the correlation between different metrics in the system. This approach is more accurate since previous forecasting techniques ignore the important characteristics (i.e. changing branching probabilities) of the deployed process instance at hand.

The remainder of the paper is structured as follows. Section 2 describes briefly the prediction methods employed in this study and introduces the key technologies used in our study: BPMN and BPEL; Section describes briefly the architecture; Section 4 presents the methodology of applying linear regression and ARIMA forecasting techniques on the estimated data; Section 5 describes a case study and preliminary results of the new architecture; Finally, the conclusions and further work are presented in Section 6.

2. BACKGROUND AND RELATED WORK

2.1 Business Process Management

A business process is collection of interrelated activities that function in a logical sequence with the ultimate focus on meeting the needs of the customer by delivering a good or service. Business Process Management (BPM) is a management approach focused on systematically organizing, managing, analyzing, and optimizing all aspects of an organization with the wants and needs of the clients. Business process management attempts to improve processes continuously, iteratively and incrementally tuning the business processes. Software tools supporting BPM throughout the business process improvement life cycle are called Business Process Management Systems (BPMS).

The BPM life cycle consists of four phases:

- *Model*. Graphical standards in this stage (such as BPMN or UML AD) allow developers, analysts and business managers to express and understand business processes and their possible flows in a diagrammatic way at a higher level [15].
- *Assemble*. At this stage, the model designed in the previous stage is assembled, developed, configured and tested on the underlying system infrastructure [15].
- *Deploy*. The *Deploy* phase is where the modeled business process is deployed on an actual BPMS engine. BPEL, the most influential execution standard in the industry, is used at this stage [15] together with the configuration from the *Assemble* phase.
- *Manage*. Business processes, after being deployed, are monitored and analyzed with appropriate tools for potential bottlenecks in order to prepare the grounds for the new design phase adaptations in response to the changing environments.

Business Process Modeling Notation (BPMN) gained much acceptance in the industry and many of its implementations can be mapped directly to BPEL code. As a core enabler of BPM, it provides the graphical notation for specifying business processes in Business Process Diagrams (BPD), based on a flowcharting technique very similar to activity diagrams. Business processes can be modeled at different levels of detail, from the perspective of all business stakeholders, which helps reduce the communication gap that frequently occurs between the *Model* and *Assemble* phases. BPMN uses a small set of graphical elements divided in five categories: flow objects (for example events, activities, gateways), data (e.g., data objects, data stores), connecting objects (such as sequence flow, message flow, association), swimlanes (e.g., pools, lanes) and artifacts (e.g., group, text annotation).

Business Process Execution Language (BPEL) for Web Services [2], also known as WS-BPEL or BPEL4WS, is an OASIS standard executable language for business processes. It allows composition of web services and is thus the top-down approach to SOA. BPEL follows the orchestration paradigm, where the involved web services do not know (and do not need to know) that they are involved in a composition and that they are part of a higher business process. BPEL is a relatively simple and straightforward way to compose these functionalities in the right order and to modify them, which makes it a more flexible approach compared to choreography. BPEL can express loops and conditional behavior, declare variables, copy and assign values, define fault handlers, and so on. By combining all these constructs, BPEL can define complex business processes in an algorithmic manner. BPEL is not as powerful as Java™, but on the other hand, it is simpler and better suited for business process definition.

2.2 Simulation

When mistakes are discovered late, it is usually difficult and expensive to correct them [18]. Investigating system behavior using simulation is more flexible, cheaper and less laborious than experimenting on the real system. The simulation model is an abstraction of a system that avoids unnecessary detail, but it can usually answer any questions about the real system [22], given that the state space of the model resembles that of the real system. BPMS monitoring capabilities make simulation easier and more reliable by providing accurate insights about the modeled system and by helping the user focus on the relevant choices that can be made [22]. Simulation models are used because they are fast, they can capture the random behavior of the resources in the system and they can be used as a decision support tool for continuous improvement or adaptation to new competitive conditions [19].

2.3 Prediction

The effectiveness of managerial decision making response to variation in the environment depends on the extent to which it can reduce the impact of uncertainty through forecasting [25, 8, 21, 20, 17, 5, 6, 7]. It is desirable to be able to predict long term change to enable more effective planning and strategic decisions, but in practice this is highly inaccurate and therefore unreliable data.

Leitner et al. [17] investigated an approach for QoS forecasting and the prediction of runtime SLA violations based on machine learning regression techniques. The described system architecture exploits a black-box prediction model, trained using historical process instances, but it does not include a simulation model. Similarly, Cavallo et al. [7] reported an empirical evaluation and comparison of different QoS prediction models. The results of their study show that an architecture that uses an ARIMA time series forecasting techniques, with no underlying simulation model, is able to predict likely QoS constraints violations.

Time series represent measurements taken sequentially over time by observing a particular phenomenon. Business process service components performance can be recorded by a monitor tool as time series data and successive observations may be regarded as dependent or partly dependent. Time series data may indicate a trend behavior over time (long term), cyclic (seasonal) or aperiodic patterns facilitating the understanding of the underlying process. The goal of short term predictive modeling is to predict the next most likely realization of the process based on the recently observed behavior. Two methods that can be used to do so are the regression method and the Box-Jenkins method. A brief description of the two follows.

2.4 Linear Regression

Regression analysis is a tool that takes the functional or statistical relationship between variables so that one can be predicted from the others.

The first order or simple linear regression model is:

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i \tag{1}$$

In this model, the intercept β_0, the slope β_1, and residual term ϵ_i are estimated parameters while y_i and x_i are measured values. Estimating β_0 and β_1 is equivalent to estimating where the regression line is located. The best estimate regarding the location of this line depends on the nature of the noise term ϵ_i. The usual assumption is that the noise term ϵ_i is on average equal to zero and that it satisfies the minimum sum of squared estimated errors criterion.

2.5 ARIMA - Auto-Regressive Integrated Moving Average

ARIMA models [3] are traditionally used for prediction of industrial and economic time series. They can identify complex temporal behavior, periodic or aperiodic, stationary or non-stationary patterns. They proved to be valuable for their ability to accurately predict short term trends based on past time series values.

The Box-Jenkins ARMA model assumes that the series under investigation is stationary. A series is stationary if its statistical proprieties (i.e. mean, variance, autocorrelation structure) do not change with time. A non-stationary series needs differencing one or more times to achieve stationarity, producing an ARIMA model.

The Box-Jenkins ARMA(p, q) model is a combination of the AR and MA models where p is the order of the autoregressive part and q is the order for the moving average part. It is expressed as a linear combination of past values of y_t and errors ϵ_t. The assumption is that current performance of the system y_t is related to the most recent p values y_{t-1}, ..., y_{t-p}.

The objective of time series analysis is to build an appropriate model for a given time series data. An ARIMA(p, d, q) model building process is done iteratively in three steps:

- *model identification* using the autocorrelation and partial autocorrelation to find the p-th and q-th orders for the auto-regressive (AR) and moving average (MA) processes. Also, it is determined if the series is stationary and if there is any significant seasonality that needs to be modeled. If the series is non-stationary differencing is applied. If seasonality is found in the data, then the length of seasonality is extracted.

- *parameter estimation*: model parameters are estimated to fit the model to the given data. Most commonly used parameters estimation methods for ARMA processes are the Hannan-Rissanen procedure[10] or Maximum Likelihood.

- *model verification* based on the estimated parameters and the model structure. By analysing the residuals of the model, it is accepted or rejected and the model building process might need to be repeated from one of the previous steps.

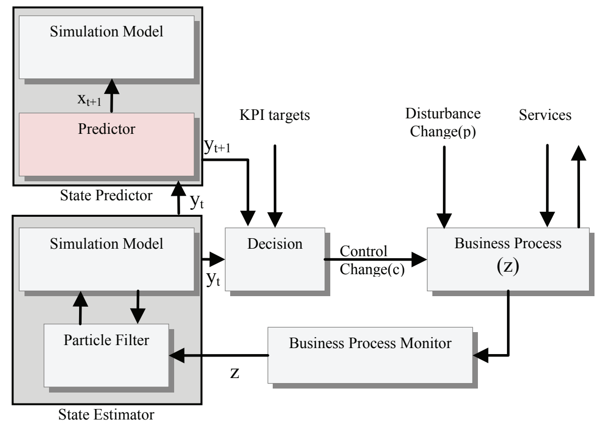


Figure 1: The proposed predictive optimization loop for a business process.

The model identification step creates often multiple competing candidate models with different numbers of parametrs for a given time series. A selection mechanism has to decide upon the final model. The Maximum Likelihood estimation method prefers the models with most parameters. Increasing the order of the model also increases the posibility that the model overfits the data, being unable to distinguish between encoded information (regularities) and noise. In response to that, several selection criteria have been developed. Akaike's Information Criterion (AIC)[1], Schwarz's Bayesian Information Criterion (BIC)[23] and Hannan-Quinn (HQ)[9] are popular criteria that measure the lack of model fit and penalize the increasing number of parameters. AIC, BIC and HQ differ only with respect to the penalty terms. The model that minimizes the criterion score is selected. As described before, another popular method of selection of the appropriate orders (p, q) for an ARMA model is known as the Hannan-Rissanen procedure[10].

3. A BUSINESS PROCESS EVOLUTION ARCHITECTURE

We propose an evolution architecture based on a feedback loop shown in Figure 1. The architecture is based on a simulation model, an estimator and predictive algorithms.

A deployed business process makes use of service components, such as web-services, that can experience external disturbances (i.e. varying arrival rates). The process and its sub-components are monitored by the Business Process Monitor and recorded for future analysis. This component also provides real time access to current values of business performance indicators (KPIs). System's goal is to maintain performance of the process close to the reference KPI targets while optimizing the use of resources. The performance, which might vary because of disturbances, is monitored and fed back through a *state estimator*. The state estimator output reflects estimated system variables, such as individual web service durations, throughputs or branching probabilities. A *particle filter*) iteratively generates simulator inputs until the simulated KPIs match the monitored ones. The estimated values, collected as a time series are then used by a state *predictor* to forecast the inputs of the simulator which in turn outputs the forecasted KPIs. In addition to the estimator and predictor, the feedback loop has a *decision block* runs a control algorithm that uses the current and predicted estimated process performance configuration to decide upon a suitable correction to bring the monitored performance closer to the desired KPI targets [4].

We use the following notation in Figure 1 and the following sections:

- $\overline{\mathbf{x}_t}$ is the *state variable* a time t. It includes variables such as individual task execution times and branch probabilities and it is extracted from $\overline{\mathbf{z}}$ by filtering out the queuing delays with a particle filter.
- $\overline{\mathbf{x}}$ is the *history vector* of estimates and contains all state variables $\langle \overline{\mathbf{x}_1}, \ldots, \overline{\mathbf{x}_t} \rangle$.
- $\mathbf{y}_t = \langle \overline{\mathbf{x}_t}, \overline{e_t} \rangle$ is the *KPI vector* at time t. The second component $\overline{e_t}$ is the end-to-end simulated process duration using the $\overline{\mathbf{x}_t}$ vector as input.
- $\overline{\mathbf{z}}$ is the *vector of measured (or observed) variables* in the process. It includes the individual task durations which is the sum of the execution times ($\overline{\mathbf{x}}$) and random variables.

4. STATE AND KPI PREDICTION METHODS

Forecasting the key performance indicators(y) to support more efficient business process optimization decisions requires a predictive model. In this section a new approach for automatic building of a dynamic predictive model is investigated. Box-Jenkins and linear regression forecasting techniques are applied on past system's state estimates generated using the proposed architecture in order to predict future process states. The business process structure is overlooked in the literature, and forecasting models often fail to produce adequate predictions because they use simple interpolation, with no correlation between different performance metrics of the system. The future behavior of the business process is reflected more accurately when simulating the forecasted estimated state and therefore it can used for dynamic, real-time business process optimization.

Once the execution of the particle filter algorithm ends, the current estimated state at step t for every task $\overline{x_k}, 1 \leq k \leq n$, n is the total number of tasks, is inserted into a state vector $\overline{\mathbf{x}_t}$ that contains all the estimates from each task. A business process execution on the server at time step t is monitored and estimated immediately. The estimation $\overline{\mathbf{x}_t}$ is inserted at the end of a history vector of estimates $\overline{\mathbf{x}} = \langle \overline{\mathbf{x}_1}, \ldots, \overline{\mathbf{x}_{t-1}} \rangle$ to become $\langle \overline{\mathbf{x}_1}, \ldots, \overline{\mathbf{x}_{t-1}}, \overline{\mathbf{x}_t} \rangle$.

4.1 Prediction - Linear Regression(LR)

When applying linear regression on the estimated data, we take into account only the most recent history with the aid of a sliding window approach. At time step t, a sliding

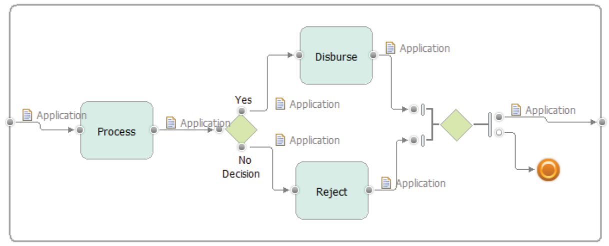


Figure 2: Business process used for experimentation.

window W_{LR} of fixed size $|W_{LR}|$, contains all the information, for each task k, $\langle \overline{\mathbf{x}_{t-|W_{LR}|+1}}, \ldots, \overline{\mathbf{x}_t} \rangle$ from $\overline{\mathbf{x}}$. It is essentially a matrix with n columns, one for each task, and $|W_{LR}|$ rows, one for each time step. By taking all the values in chronological order on the kth column, where $1 \leq k \leq n, n$ is the total number of tasks, linear regression is applied and $\boldsymbol{\beta}_{k,t} = \langle \beta_{0,k}, \beta_{1,k} \rangle_t$ coefficients for each activity k are estimated. Using the intercept $\beta_{0,k}$ and slope $\beta_{1,k}$, the duration at time step $t+1$ can be predicted.

4.2 Prediction - ARIMA

In order to construct an online model to predict task service time patterns, sliding windows are often used to create and adapt the model. However, we develop an ARIMA model by examining the entire data for model creation and validation. Therefore, instead of having a fixed size, the window is a matrix with n columns and t rows, to contain the entire history. A total number of n ARIMA models are built at each time step t, one for each column. To determine the appropriate Box-Jenkins model, the Hannan-Rissanen model identification procedure[10] is used.

5. CASE STUDY

This section describes the way we apply our methodology on a deployed business process and how we investigate the predictive improvement of our state estimation model, by comparing the current forecasting methods and our proposed ARIMA technique. The goal of this case study is to show that a simulation model can capture the correlation between metrics and provide more reliable and accurate predictions in comparison with today industrial prediction techniques where no simulation model is employed. Based on these results, an ARIMA model together with a simulation model are proposed for future research in BPM.

5.1 Infrastructure

We used IBM® WebSphere Business Modeler Advanced® [13] to design, simulate and analyze a business process under a wide variety of conditions. Modeler can also transform the process into BPEL for implementation, generate reports and redefine the existing and proposed processes together with the required resources and business items. For monitoring, we used WebSphere Business Monitor® [11] on a WebSphere Process Server® [14]. It is an application that is deployed on a server to collect business performance data during the BPM mange life-cycle phase and to provide it for analysis via dashboards or REST APIs to the user. Built on top of WebSphere Application Server®, WebSphere Process Server® supports traditional business integration and advanced business process solutions based on service-oriented architecture (SOA). Finally, WebSphere Integration Developer® [12] was used for assembling (assemble BPM life-cycle phase) and deployment (deploy BPM life-cycle phase) of business processes. WebSphere Integration Developer® is a tool for business process integration, allowing for flexible service components wiring and rapid composite application development [27]. On top of these tools, we integrated JMulti [16] package for (univariate) ARIMA model [3] specification, estimation, selection, checking and forecasting.

5.2 Workloads

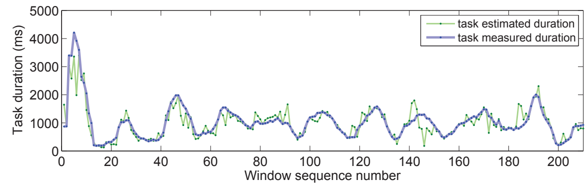
We used for experimentation a simple business process containing three tasks and a decision node, similar to the one in Figure 2. Each task is automated by a web-service deployed on a server. Real workloads were generated in order to observe how both the state estimator and state predictor respond to load variation. Variations in measurements at the monitor side were observed by sending multiple requests at the same time as a bundle and multiple bundles at random time intervals. The measurements reflect multiple regions of increased workload, generated by bundled requests arriving simultaneously or at short inter-arrival times. Generated workloads were designed to test the accuracy of the two prediction methods in various scenarios of variable arrival rate with uniform service time per web-service. The aggregated values were calculated using a sliding window of size 12. Variable window sizes can be used since this is a tuning parameter for the state estimator.

5.3 Results

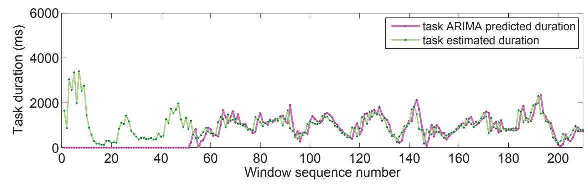
The focus is on the short term predictions, $t+1$, because in general they are more accurate and reliable than medium and long term predictions. Therefore, we only use several hundred workload measurements to examine the performance of the two predictive models.

When doing real-time monitoring both windows W_{LR} and W_{ARIMA} are initially empty. As the business process is executed and tasks executions are measured, the two windows accumulate estimation data $\overline{\mathbf{x}}$ in parallel. In comparison with the ARIMA approach, linear regression requires fewer data points to start. For instance, in Figure 3(c) linear regression predicted durations lag just a few instances behind the first estimates. On the other hand, in Figure 3(b) the Box-Jenkins method needs about an order of magnitude greater than the minimum number of estimated values necessary for the linear model. Figure 3(a) compares, for a specific task, the aggregated monitored data and its estimated values that exits the state estimator. The following two figures, 3(b) and 3(c), use the estimated states in Figure 3(a), as reference for plotting the predicted values of the two methods.

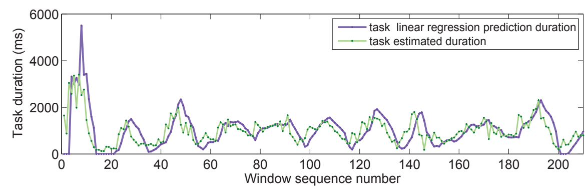
We consider three cases, illustrated in Figure 4 where we apply the two forecasting methodologies. In Figure 4 (a), at time step t, we feed all window state estimates $\overline{\mathbf{x}_i} \in W_{LR}, t-|W_{LR}|+1 \leq i \leq t$, straight to the linear regression module in order to find all n tasks performance forecasts future estimated state in $\overline{\mathbf{x}_{t+1}}$. They are simulated once to predict the average end-to-end duration $\overline{e_{t+1}}$ of the process. A vector $\mathbf{y}_{t+1} = \langle \overline{\mathbf{x}_{t+1}}, \overline{e_{t+1}} \rangle$, encapsulates the state



(a) Task aggregated and estimated durations.



(b) Task ARIMA predicted durations based on estimations.



(c) Task linear regression predicted durations based on estimations.

Figure 3: State estimation and prediction using linear regression and ARIMA.

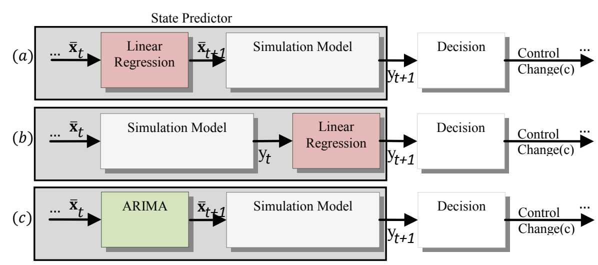


Figure 4: Prediction methods comparison.

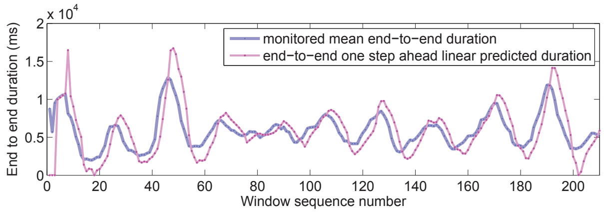
estimates $\overline{\mathbf{x}_{t+1}}$ at time step $t+1$ and their simulated end-to-end duration $\overline{e_{t+1}}$.

The second case, in Figure 4 (b), all estimates $\overline{\mathbf{x}_t} \in W_{LR}$ are simulated producing a second window $W^e_{LR} = \langle \overline{e_{t-|W_{LR}|+1}}$ $, \ldots, \overline{e_t} \rangle$ of end-to-end durations. By passing this linear regression window forward to the regression module, we obtain $\overline{e_{t+1}} \in \mathbf{y}_{t+1}$.

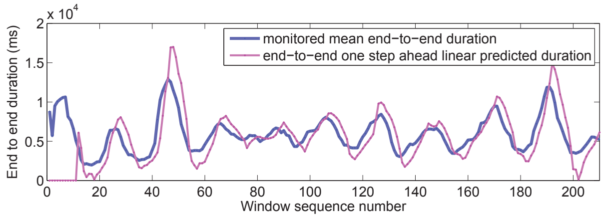
The last case of forecasting $\mathbf{y}_{t+1}$ is depicted in Figure 4 (c). This case is similar to the first one, in Figure 4 (a), but instead of using a linear regression module and W_{LR} we use the ARIMA module described in section 4.2 together with a history window W_{ARIMA}.

Figures 5(a), 5(b) and 5(c) present the forecasted mean end-to-end durations, produced by prediction methods (a), (b) and (c) respectively, in comparison with the mean end to end monitored duration.

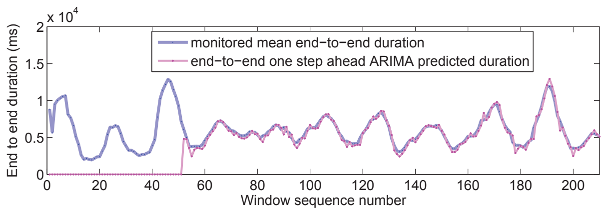
Three measures for forecast accuracy were used: Mean Absolute Percentage Error (MAPE) and Worst Absolute



(a) Prediction in case (a).



(b) Prediction in case (b).



(c) Prediction in case (c).

Figure 5: Business process end-to-end prediction and errors using case (a), (b) and (c) from Figure 4.

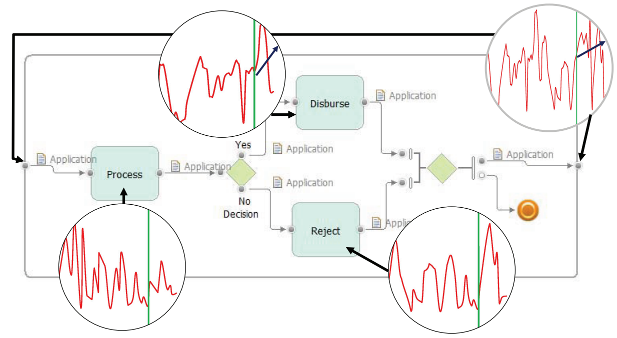
Percentage Error (WAPE).

$$MAPE = \frac{1}{t}\sum_{i=1}^{t}(\frac{|predicted\ \mathbf{e}_i - measured\ \mathbf{e}_i|}{measured\ \mathbf{e}_i}) \quad (2)$$

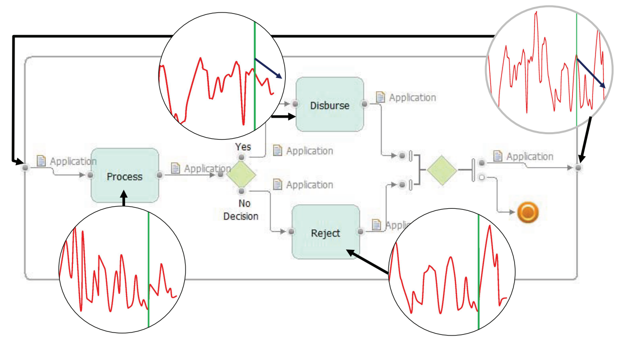
$$WAPE = max(\frac{|predicted\ \mathbf{e}_i - measured\ \mathbf{e}_i|}{measured\ \mathbf{e}_i}) \quad (3)$$

By computing $MAPE$ for each case we find that prediction method (a) with a $MAPE = 21\%$ and $WAPE = 79\%$ measures a better fit of the model than method (b) with a $MAPE$ of 23% and a $WAPE$ of 96%, but the best performing method remains case (c), with only 7% $MAPE$ and 45% $WAPE$.

The conclusion of these experiments is that the use of Box-Jenkins ARIMA (p, d, q) models on stationary or non-stationary estimated metrics is beneficial because they can capture regularities and irregularities in the data. At the same time they provide good input for the predictive simulation process to identify more accurate short term KPI trends and the opportunity to optimize a business goal by re-allocating resources dynamically. Also $MAPE$ and $WAPE$ measures, for linear regression method that takes into account the structure of the process, suggest that the error is more stable and less significant when studied in relation to the size of the actual measurement. A prediction model without simulation would be unable to distinguish between case (a) and case (b) in Figures 6(a) and 6(b) respectively. Both cases present the execution of the business process and their estimated performance at each web-service. Their execution is identical on the performance graphs up to the green vertical line (the current point in time). Data located



(a)



(b)

Figure 6: Simulating forecasted values.

on the left and right of the green line represent past and future estimations of the system. Observe that, in a simple situation where only one web-service behaves differently in both cases, and the blue trend arrows are detected as going up or down, a simulation model would be able to explain why end-to-end duration of the system is degrading or not, via simulation reports. Without simulation, these cases are indistinguishable and therefore it translates into lost opportunities for correction or dynamism.

6. CONCLUSIONS AND FUTURE WORK

To summarize, a new approach for automatic building of a dynamic predictive model was investigated to support more efficient business process optimization decisions. We built a predictive model and we compared linear regression and Box-Jenkins forecasting techniques using two forecast accuracy measures: $MAPE$, $WAPE$. The experimentation results suggest that simulating predicted states for a more confident and consistent prediction is an advantage and also that ARIMA models are good candidates for future application and research in BPM.

7. ACKNOWLEDGMENTS

This research was supported by OCE, the Ontario Centres of Excellence and by the IBM Toronto Centre for Advanced Studies, as part of the program of the Centre for Research in Adaptive Systems (CERAS).

8. REFERENCES

[1] H. Akaike. Fitting autoregressive models for prediction. *Annals of the Institute of Statistical Mathematics*, 21(1):243–247, 1969.

[2] T. Andrews, F. Curbera, H. Dholakia, Y. Goland, J. Klein, F. Leymann, K. Liu, D. Roller, D. Smith, and S. Thatte. Business process execution language for web services specification, version 1.1. *IBM, Microsoft, BEA, SAP and Siebel Systems*, 2003.

[3] G. E. Box and G. Jenkins. *Time Series Analysis, Forecasting, and Control*. Holden-Day, San Fransisco, 1970.

[4] Y. Brun, G. D. M. Serugendo, C. Gacek, H. Giese, H. Kienle, M. Litoiu, H. MÃijller, M. PezzÃÍ, and M. Shaw. Engineering self-adaptive systems through feedback loops. *Software Engineering for Self-Adaptive Systems*, pages 48–70, 2009.

[5] S. Casolari, M. Andreolini, and M. Colajanni. Runtime prediction models for web-based system resources. In *Modeling, Analysis and Simulation of Computers and Telecommunication Systems, 2008. MASCOTS 2008. IEEE International Symposium on*, pages 1–8. IEEE, 2009.

[6] M. Castellanos, N. Salazar, F. Casati, and U. Dayal. Predictive business operations management. *International Journal of Computational Science and Engineering*, 2(5):292–301, 2006.

[7] B. Cavallo, M. D. Penta, and G. Canfora. An empirical comparison of methods to support QoS-aware service selection. In *Proceedings of the 2nd International Workshop on Principles of Engineering Service-Oriented Systems*, pages 64–70. ACM, 2010.

[8] A. D'Ambrogio and P. Bocciarelli. A model-driven approach to describe and predict the performance of composite services. In *Proceedings of the 6th international workshop on Software and performance*, pages 78–89. ACM, 2007.

[9] E. J. Hannan and B. G. Quinn. The determination of the order of an autoregression. *Journal of the Royal Statistical Society. Series B (Methodological)*, 41(2):190–195, 1979.

[10] E. J. Hannan and J. Rissanen. Recursive estimation of mixed autoregressive-moving average order. *Biometrika*, 69(1):81, 1982.

[11] IBM. Websphere business process monitor. *http://www.ibm.com/software/integration/wbimonitor/*, 2010.

[12] IBM. Websphere integration developer. *http://www.ibm.com/software/integration/wid/*, 2010.

[13] IBM. Websphere modeler. *http://www.ibm.com/software/integration/wbimodeler/*, 2010.

[14] IBM. Websphere process server. *http://www.ibm.com/software/integration/wps/*, 2010.

[15] R. K. L. Ko, S. S. G. Lee, and E. W. Lee. Business process management (BPM) standards: a survey. *Business Process Management Journal*, 15(5), 2009.

[16] M. KrÃatzig and H. LÃutkepohl. jmulti. *http://www.jmulti.de/*, 2009.

[17] P. Leitner, B. Wetzstein, F. Rosenberg, A. Michlmayr, S. Dustdar, and F. Leymann. Runtime prediction of service level agreement violations for composite services. In *Service-Oriented Computing. ICSOC/ServiceWave 2009 Workshops*, pages 176–186. Springer, 2010.

[18] E. Paolucci, F. Bonci, and V. Russi. Redesigning organisations through business process re-engineering and object-orientation. In *Proceedings of the European Conference on Information Systems*, pages 587–601, 1997.

[19] R. J. Paul, V. Hlupic, and G. Giaglis. Simulation modeling of business processes. In *Proceedings of the 3rd UK Academy of Information Systems Conference*, pages 311–320. Citeseer, 1998.

[20] H. A. Reijers. Case prediction in BPM systems: A research challenge. *Journal of the Korean Institute of Industrial Engineers*, 33:1–10, 2006.

[21] H. A. Reijers and W. M. P. van der Aalst. The effectiveness of workflow management systems: predictions and lessons learned. *International Journal of Information Management*, 25(5):458–472, 2005.

[22] P. J. Sanchez. Fundamentals of simulation modeling. In *Proceedings of the 39th conference on Winter simulation: 40 years! The best is yet to come*, pages 54–62. IEEE Press, 2007.

[23] G. Schwarz. Estimating the dimension of a model. *The annals of statistics*, 6(2):461–464, 1978.

[24] A. Solomon, M. Litoiu, J. Benayon, and A. Lau. Business process adaptation on a tracked simulation model. In *Proceedings of the 2010 Conference of the Center for Advanced Studies on Collaborative Research*, pages 184–198. ACM, 2010.

[25] M. Teixeira, R. Lima, C. Oliveira, and P. Maciel. Performance evaluation of service-oriented architecture through stochastic petri nets. In *Systems, Man and Cybernetics, 2009. SMC 2009. IEEE International Conference on*, pages 2831–2836. IEEE, 2009.

[26] L. Verner. BPM: the promise and the challenge. *Queue*, 2(1):91, 2004.

[27] U. Wahli, L. Leybovich, E. Prevost, R. Scher, and A. Venancio. Business process management: Modeling through monitoring (Ibm redbooks). 2006.